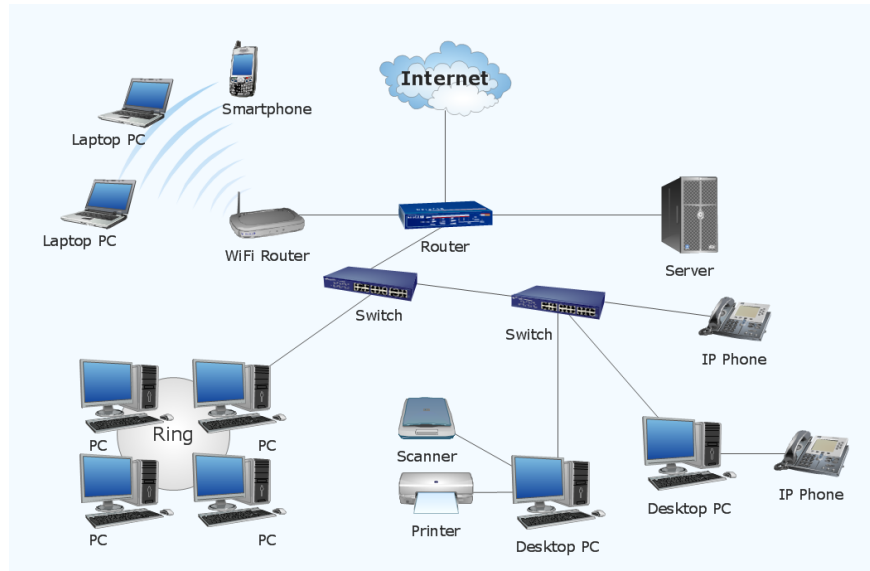


Sisteme și algoritmi distribuiți

Curs 1

f



Plan materie

1. Introducere. Modele și arhitecturi SD.

2. Comunicație în SD. Ceasuri logice.

3. Sisteme sincrone

3.1 Algoritmi OSD: alegere lider, consens, sincronizare

3.2 Defecte

3.3 Consens și alte probleme numerice: medie, evaluare funcții, sisteme liniare

3.4 Toleranță la defecte

4. Sisteme asincrone

4.1 Organizarea SDa: teorema imposibilitate, consens relaxat

4.2 Medie și sisteme liniare în context asincron

4.3 Toleranță la defecte

5. Algoritmi distribuiți aleatori:

5.1 Paradigma gossip

5.2 Consens și alte probleme

5.3 Toleranță la defecte

Organizare

Curs 2h/săptămână: A. P.

Laborator 2h/ săptămână: Marius Mihăilescu

Evaluare:

1. Proiect final (50%)
2. Teme laborator (50%)

Promovare: nota 5 examen + nota 5 laborator.

Materiale și comunicare: platforma Teams.

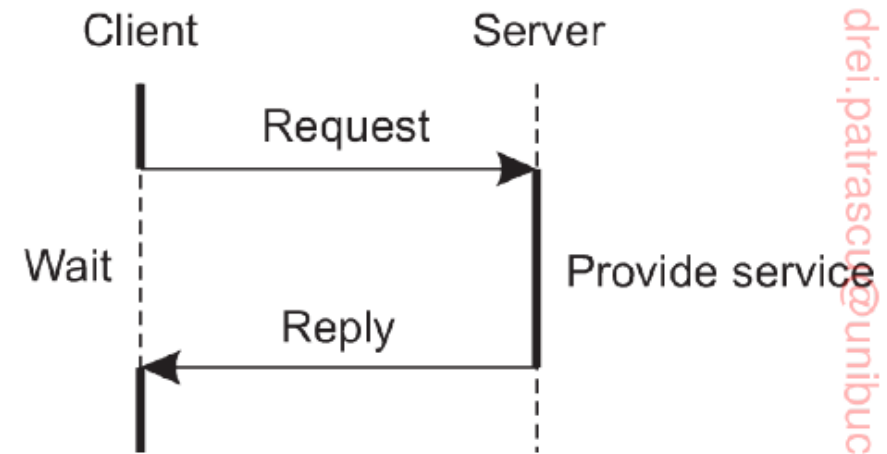
Adresa e-mail: andrei.patrascu@fmi.unibuc.ro,
marius-iulian.mihailescu@unibuc.ro

Arhitecturi: Client-Server

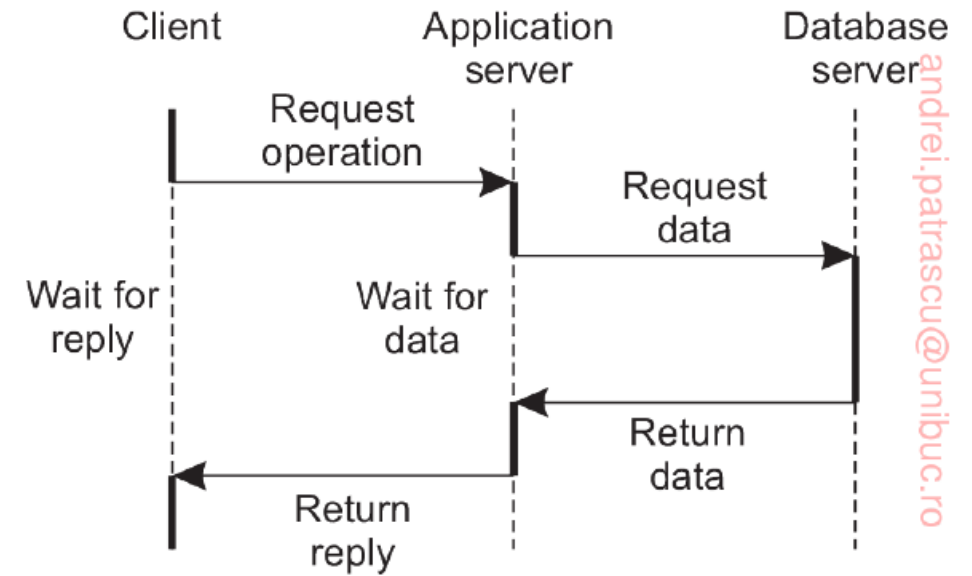
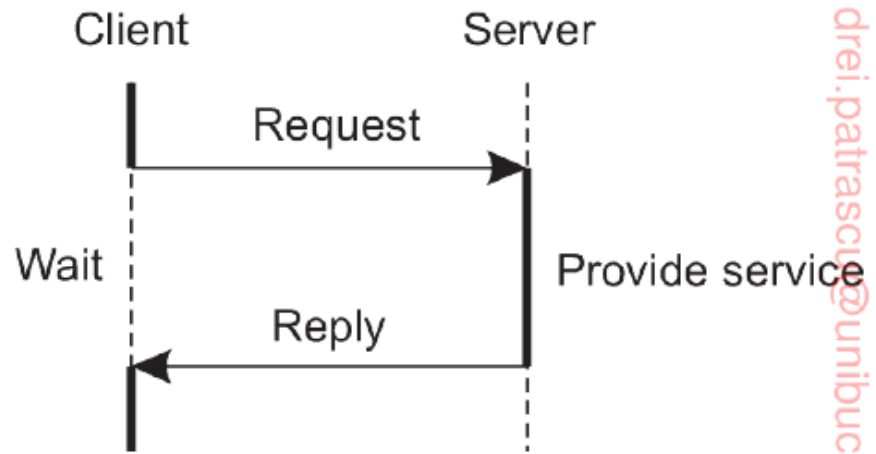
- Un nod/proces are calitatea de client sau server
- Server = entitate care ofera un serviciu
- Client = entitate care apeleaza un serviciu

Exemple:

- Microsoft Teams
- Overleaf



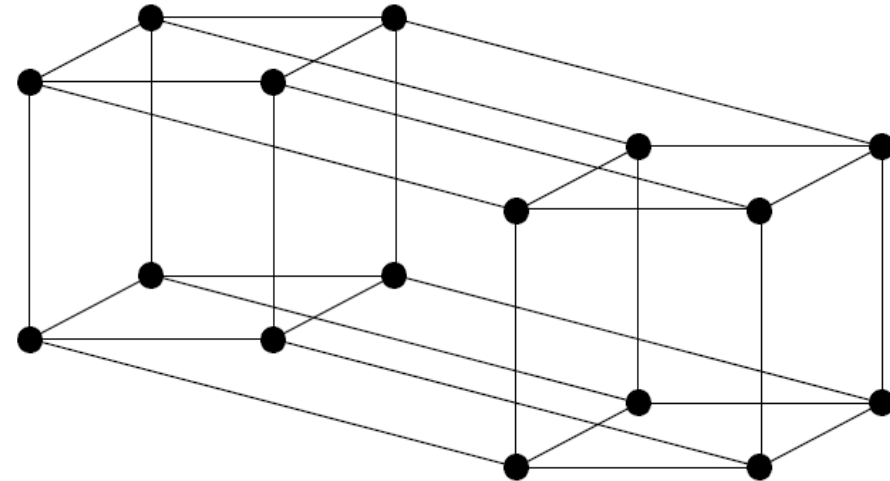
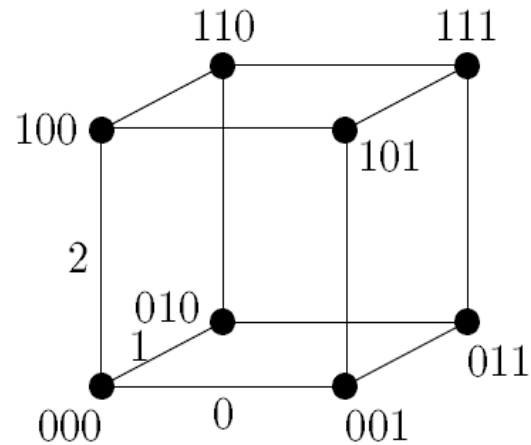
Arhitecturi: Client-Server



Arhitecturi: Peer-to-peer structurate

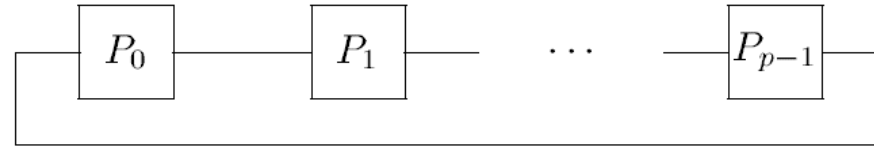
Cluster (parallel) computing:

- Fiecare nod are aceeași capacitate de calcul
- Comunicație sigură între procese
- Identificator unic per proces (nod)

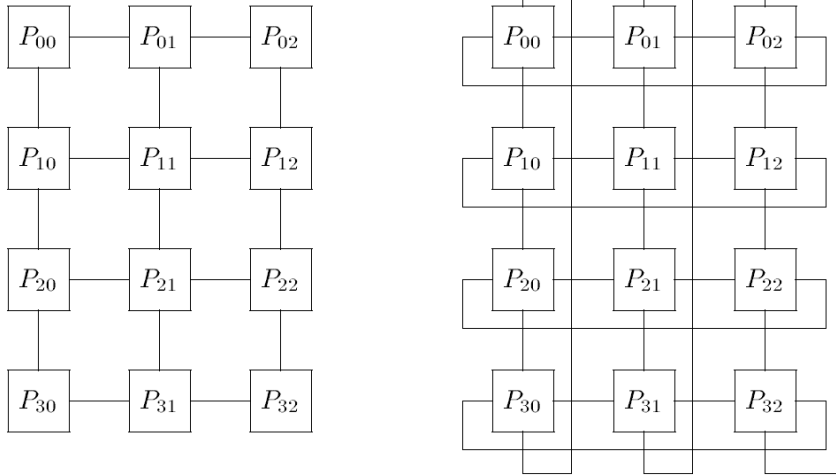


Arhitecturi: Peer-to-peer structurate

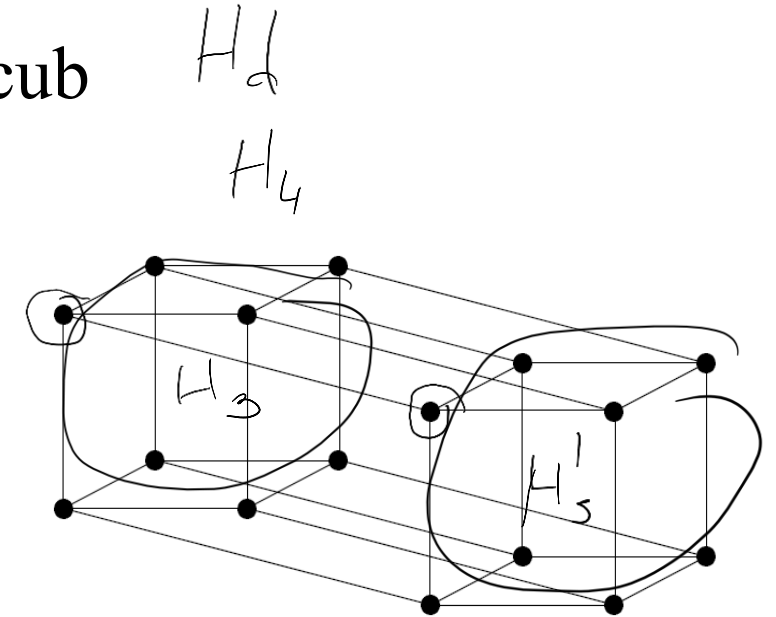
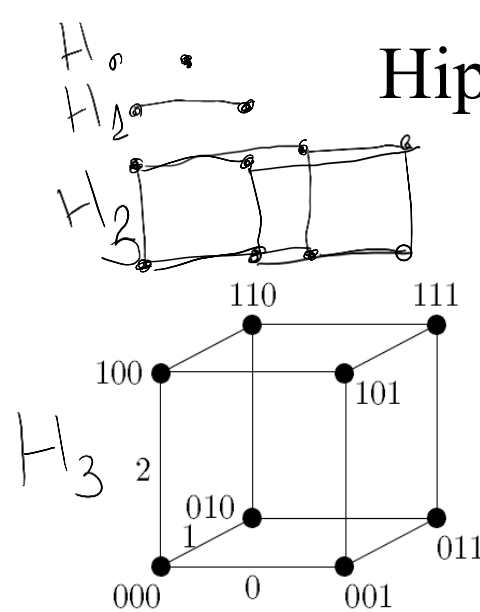
Inel



Grilă/Tor



Hipercub



Peer-to-peer structurat: El Capitan

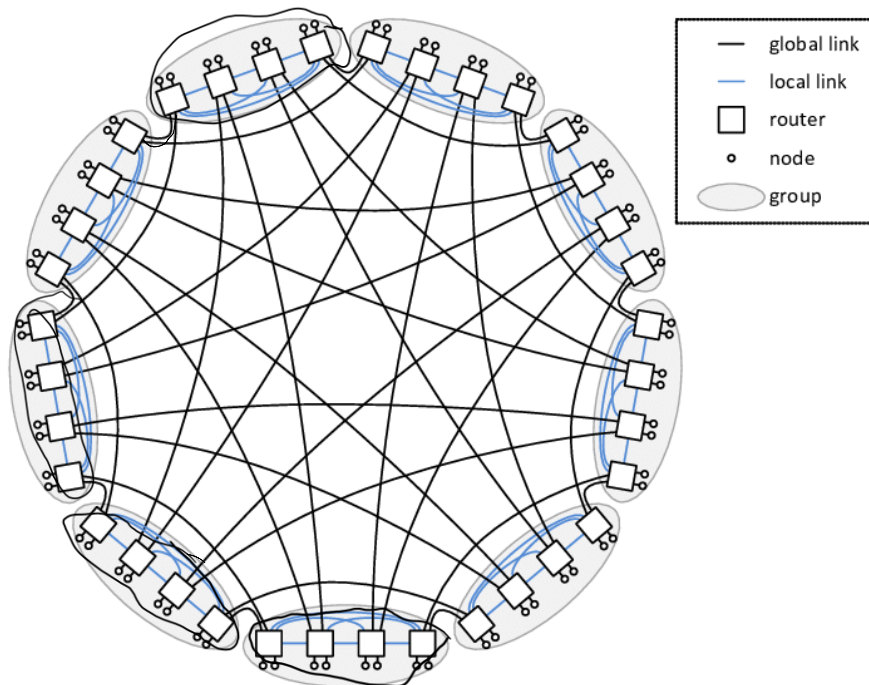
top 500, org



Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	<u>El Capitan</u> - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	<u>11,039,616</u>	<u>1,742.00</u>	2,746.38	<u>29,581</u>
2	<u>Frontier</u> - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	<u>Aurora</u> - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	<u>JUPITER Booster</u> - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	793.40	930.00	13,088
5	<u>Eagle</u> - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	

El capitan

- 43,808 CPU AMD Epyc 24C "Genoa" (24 cores)
- 43,808 AMD Instinct MI300A GPUs
 - 11.039.616 core-uri
- 87 rack-cabinets
- 1 server-blade are 2 noduri
- Nodurile sunt structurate pe grupuri; grupurile sunt conectate într-o topologie “dragonfly”



Sistem paralel (Frontier cluster)

- Informații globale disponibile: număr de noduri ale rețelei, topologia rețelei, distribuția datelor în rețea, indexarea globală a nodurilor
 - Control asupra distribuției datelor
 - Control asupra execuției locale per nod
 - Control asupra implicării nodurilor în rețea

MPI-Rank
- Timp de comunicație inter-noduri neglijabil/mărginit (apropiere geografică)
 - Sincronizare: ceas fizic comun
- Topologie statică
- Probabilitatea scăzută a defectelor
- Complexitatea timp vs. complexitatea mesaj



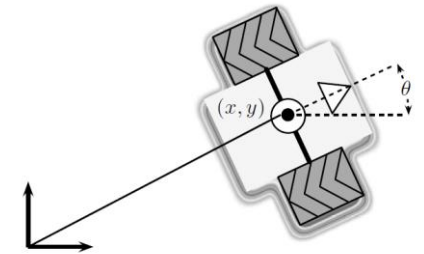
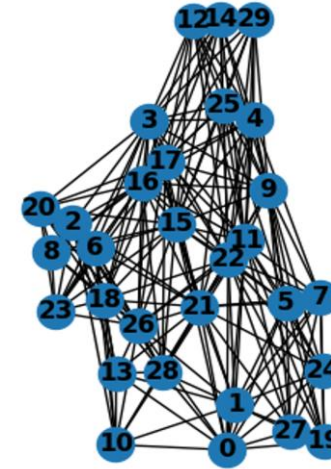
Arhitecturi: Peer-to-peer nestructurate

Rețele de roboți (agenți mobili), sisteme de informații, aplicații:

- Localizare distribuită (GPS, sau ...)
- Urmărire distribuită
- Achiziție distribuită
- Editare distribuită (colaborativă)
- Rezolvare distribuită de sisteme

Caracteristici:

- Anonimitate: nodurile nu au un identificator unic global
- Comunicația între două noduri se realizează prin muchiile disponibile
- Topologia potențial variabilă în timp
- Posibile defecte pe legături sau noduri



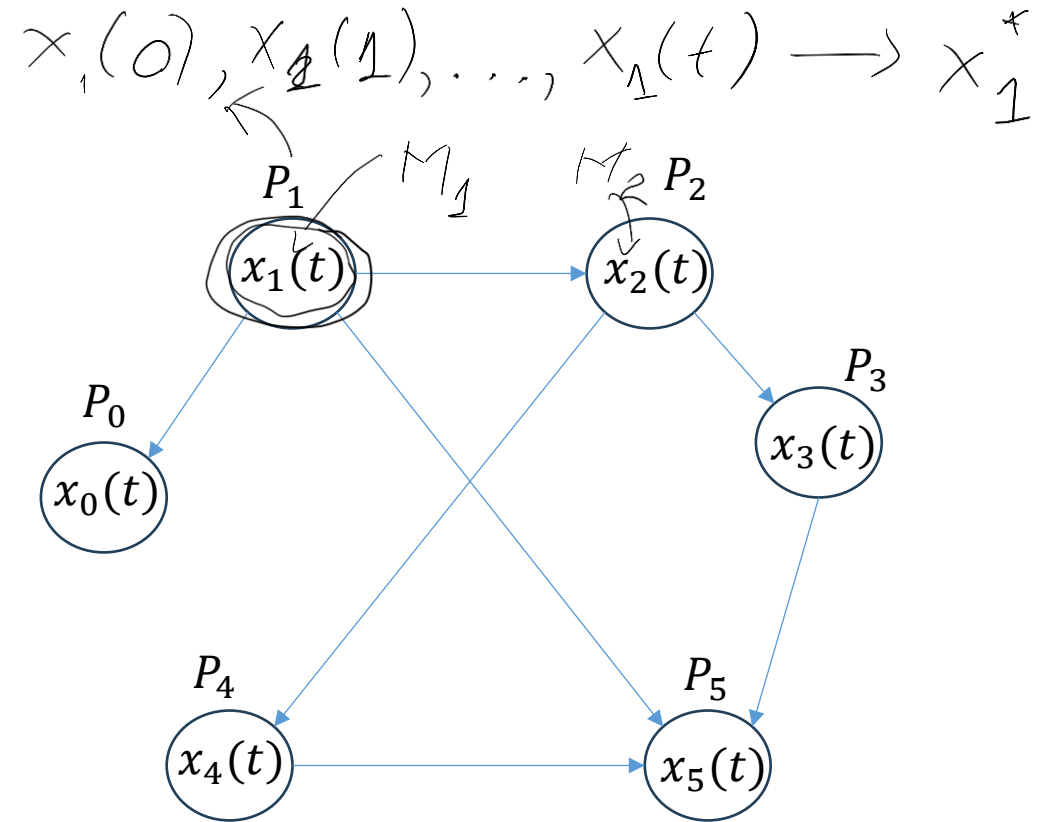
Sistem distribuit

Sistem distribuit = o colecție de procese autonome care comunică peste o rețea (topologie) cu următoarele proprietăți:

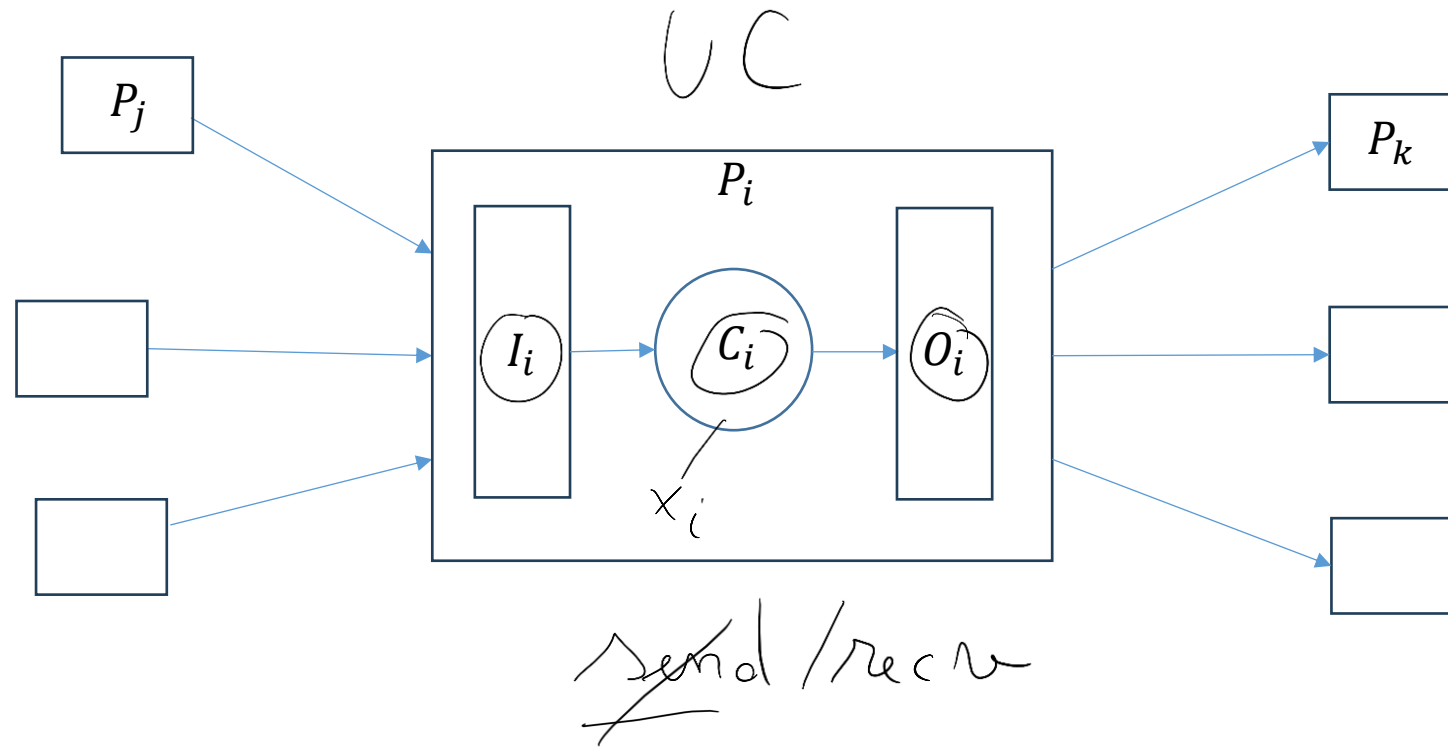
- Fiecare nod are o „vedere” locală asupra sistemului. Un nod al sistemului cunoaște și comunică cu propria vecinătate, neavând acces la informații globale. În general, există o separare geografică a nodurilor.
- Nu există un ceas fizic comun. Acestui aspect se datorează caracterul “distribuit” al sistemului și este cel care cauzează lipsa sincronizării între noduri.
- Nu există memorie partajată. Proprietate care aduce necesitatea comunicației prin mesaje (în absența unui ceas global).
- Autonomia și eterogenitatea nodurilor. Noduri sunt “slab cuplate”, au viteze diferite de execuție, au sisteme de operare diferite.

Model SD

- Noduri: $P_0, P_1, \dots, P_{n-1}$
- Rețea de comunicație $G = (V, E)$
 - $V = \{P_0, P_1, \dots, P_{n-1}\}$
 - $(i, j) \in E$ dacă există muchie între nodurile P_i și P_j
- Starea nodului P_i se exprima $x_i: \mathbb{N} \rightarrow \mathcal{D}$
- Stare nod P_i la momentul de timp $t: x_i(t)$
 - Temperatura (real)
 - Locație-Viteza (real)
 - Vot-Opinie (bimer)

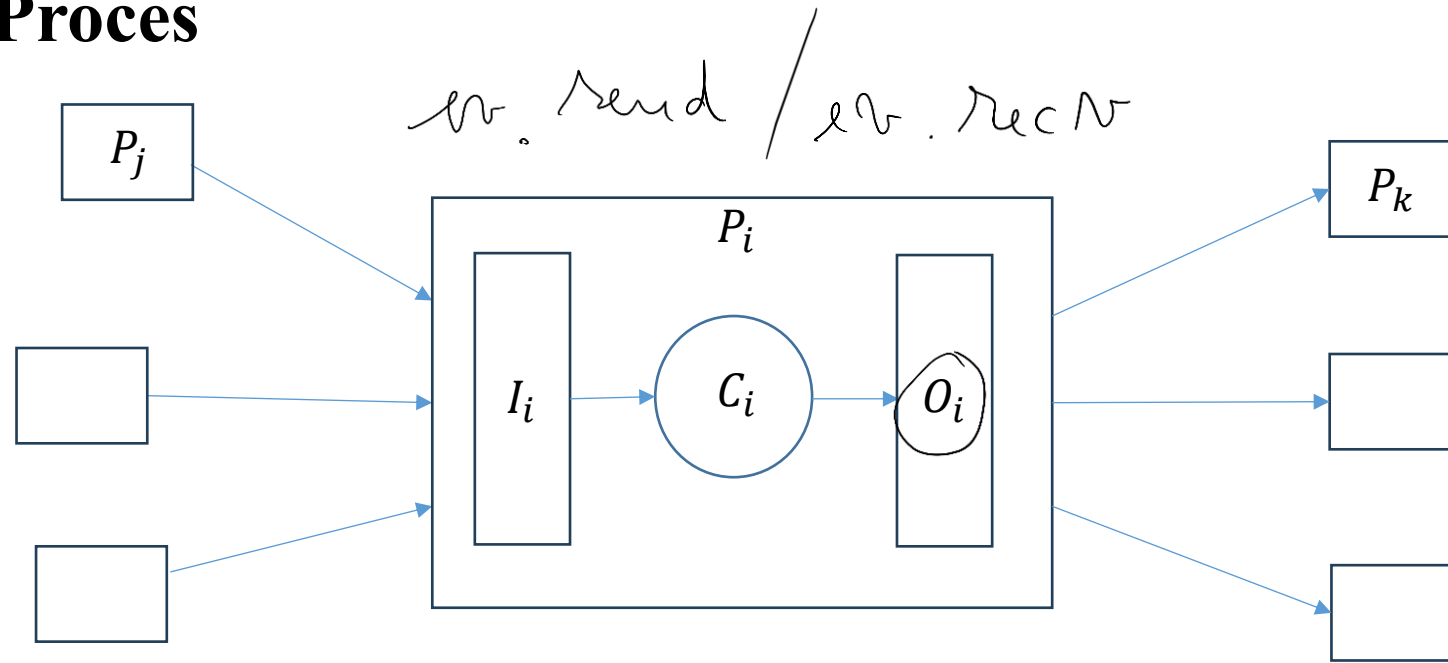


Model Proces



- Un proces primește informație la intrare (mesaje de la vecini), stocată în I_i
- Calculează noua stare pe baza info de input; în plus, depune în O_i mesaje pentru transmitere
- Transmite mesajele din O_i către vecinii de ieșire
- Transferul de mesaje este un eveniment separat reușit/esuat

Model Proces



Mai formal:

- $I_i(t), O_i(t), x_i(t)$ buffer-ele de intrare si iesire, si starea la momentul t
- Nodul i primeste mesajul m la intrare: $I_i(t+1) = I_i(t) \cup \{m\}$
- Calculeaza noua stare pe baza intrarii: $f_i(x_i(t), m) \rightarrow (x_i(t+1), \{m_1, \dots, m_k\})$
- Transmite mesajele de iesire: $O(t+1) = O_i(t) \cup \{m_1, \dots, m_k\}$
- Consideram I/O de mesaje ca evenimente separate in retele nesigure

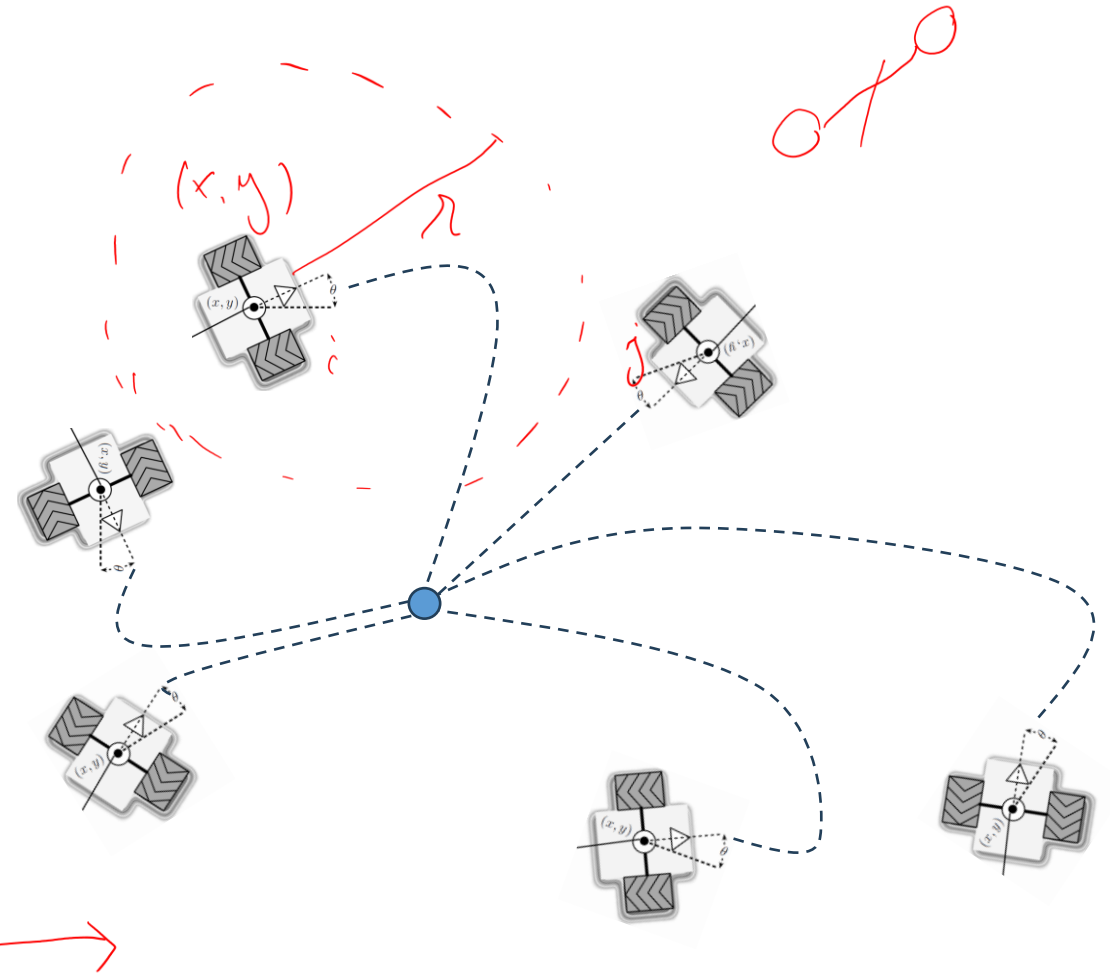
Exemplu model sincron: rendezvous 2D

Rendezvous 2D: Considerăm o rețea de n agenți mobili $\{1, 2, \dots, n\}$ cu pozițiile inițiale $\{p_1(0), p_2(0), \dots, p_n(0)\}$. Conduceți agenții prin intrări, calculate la nivel local, către un punct de întâlnire arbitrar, i.e. $p_i = p_j, \forall i, j \in \{1, 2, \dots, n\}$.

Prin notația $p(t)$ extragem poziția din vectorul de stare $x(t) \in X$, și considerăm modelul simplificat al agenților: $p_i(t+1) = p_i(t) + u_i(t)$

$$x_i(t) = \begin{bmatrix} p_x(t) \\ p_y(t) \\ \theta(t) \end{bmatrix}$$

$\in \mathbb{R}^3$

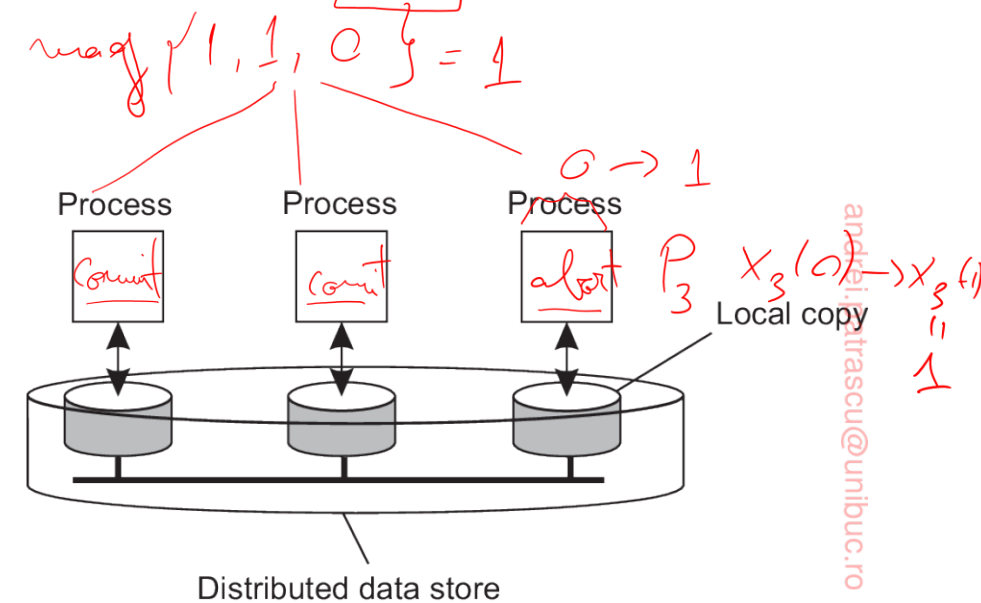


Exemplu model sincron: vot majoritar

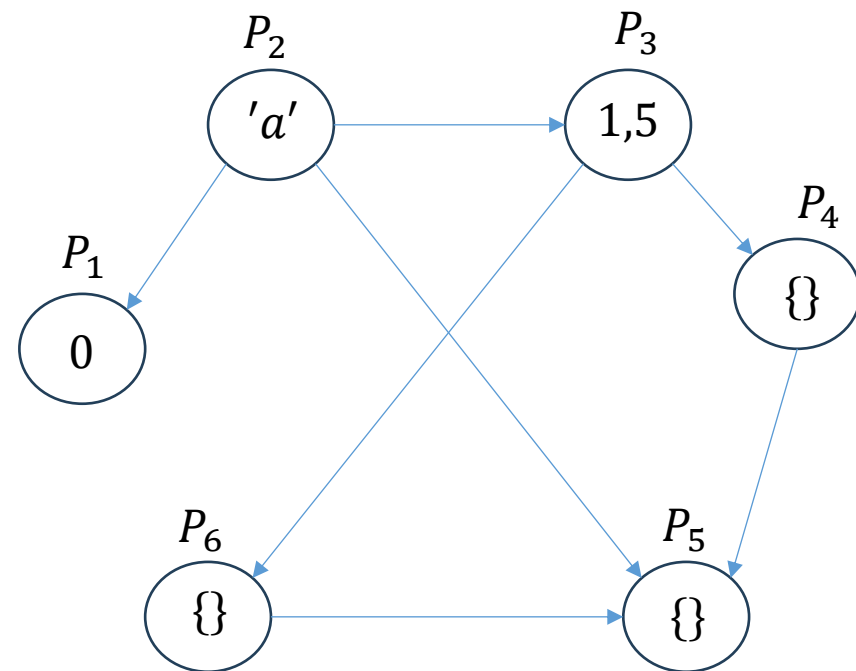
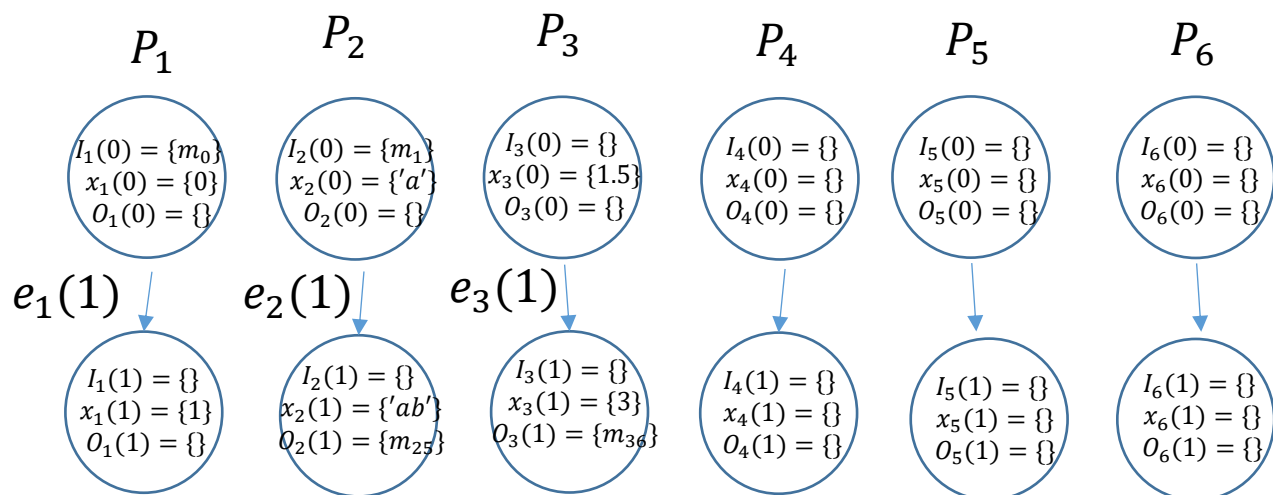
Majority voting: Considerăm o rețea de n replici de date $\{1, 2, \dots, n\}$ cu voturi inițiale $\{x_1(0), x_2(0), \dots, x_n(0)\}$. Aduceți replicile prin interacțiuni la nivel local, către valoarea asumată majoritar la nivel global

- Starea nodului denotă acțiunea de commit/abort a unei posibile tranzacții, i.e. $x_i(t) \in \{0, 1\}$

$$x_i^* = \text{Maj}(x(0)) = \begin{cases} 1, & \text{dacă } |\{i | x_i(0) = 1\}| \geq \frac{n}{2} + 1 \\ 0, & \text{dacă } |\{i | x_i(0) = 1\}| < \frac{n}{2} + 1 \end{cases}$$



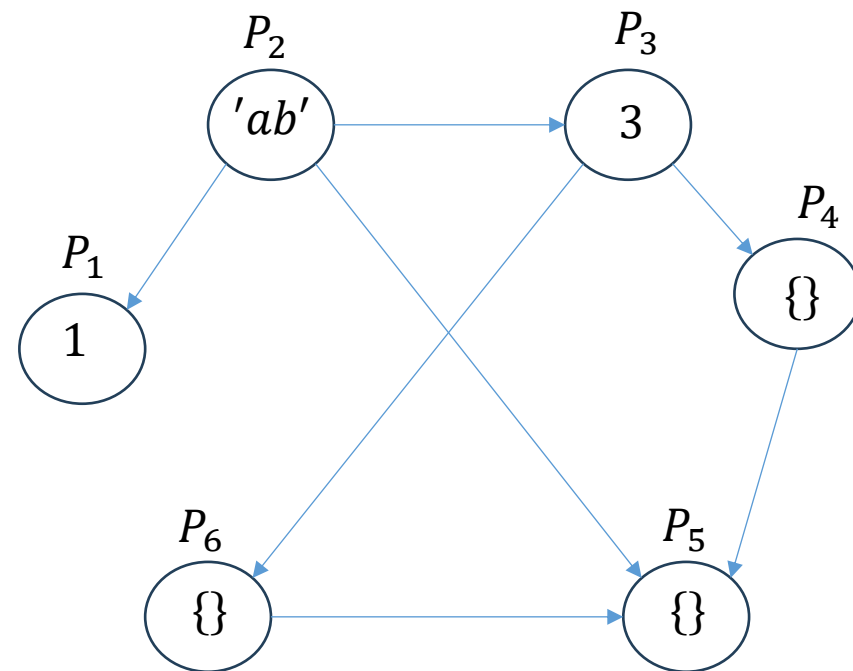
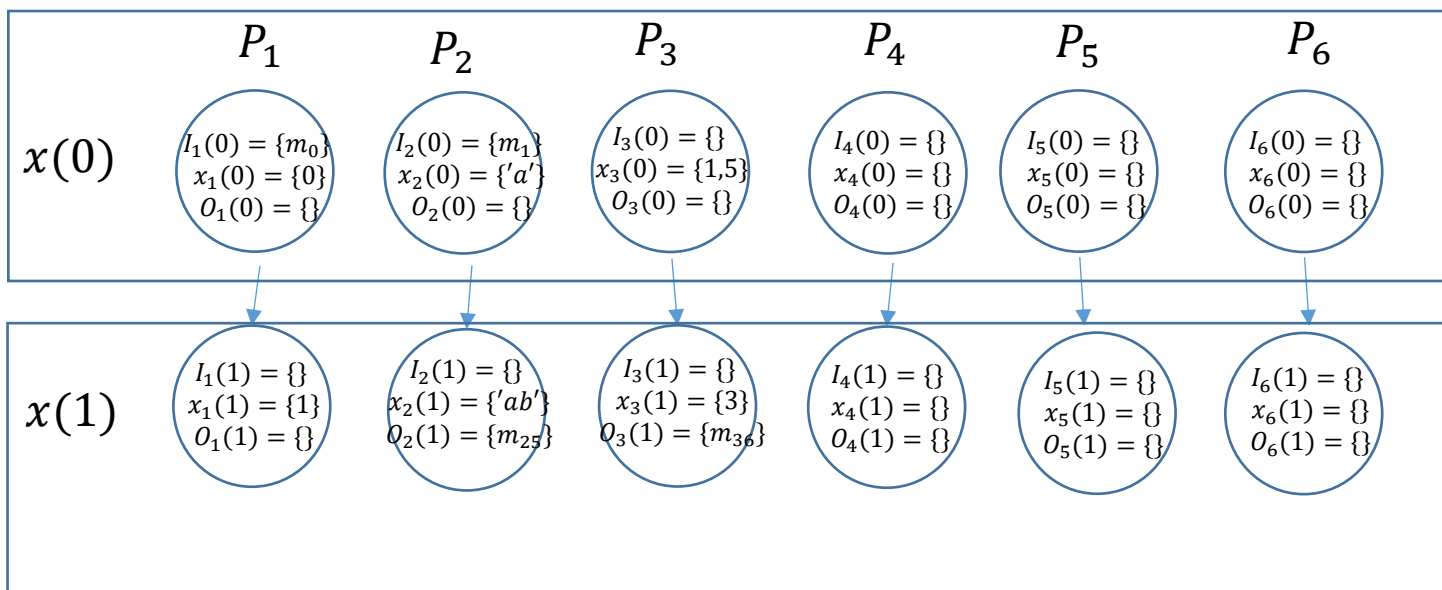
Modelul principal in SD sincron



Evenimente posibile:

- Operatii locale pe baza $I_i(t)$ si $x_i(t)$: e.g. calcule numerice
- Evenimente de livrare de mesaje

Modelul principal in SD sincron



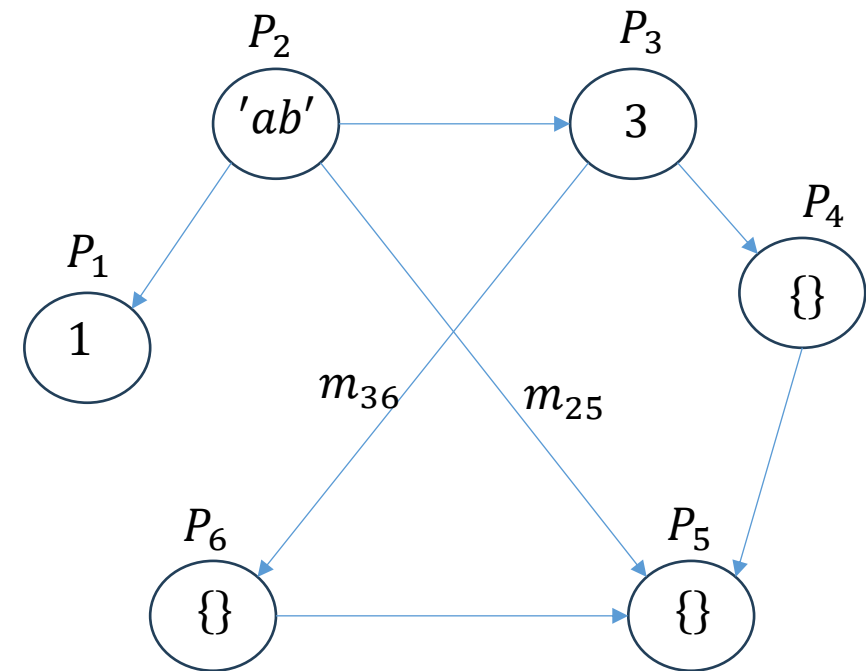
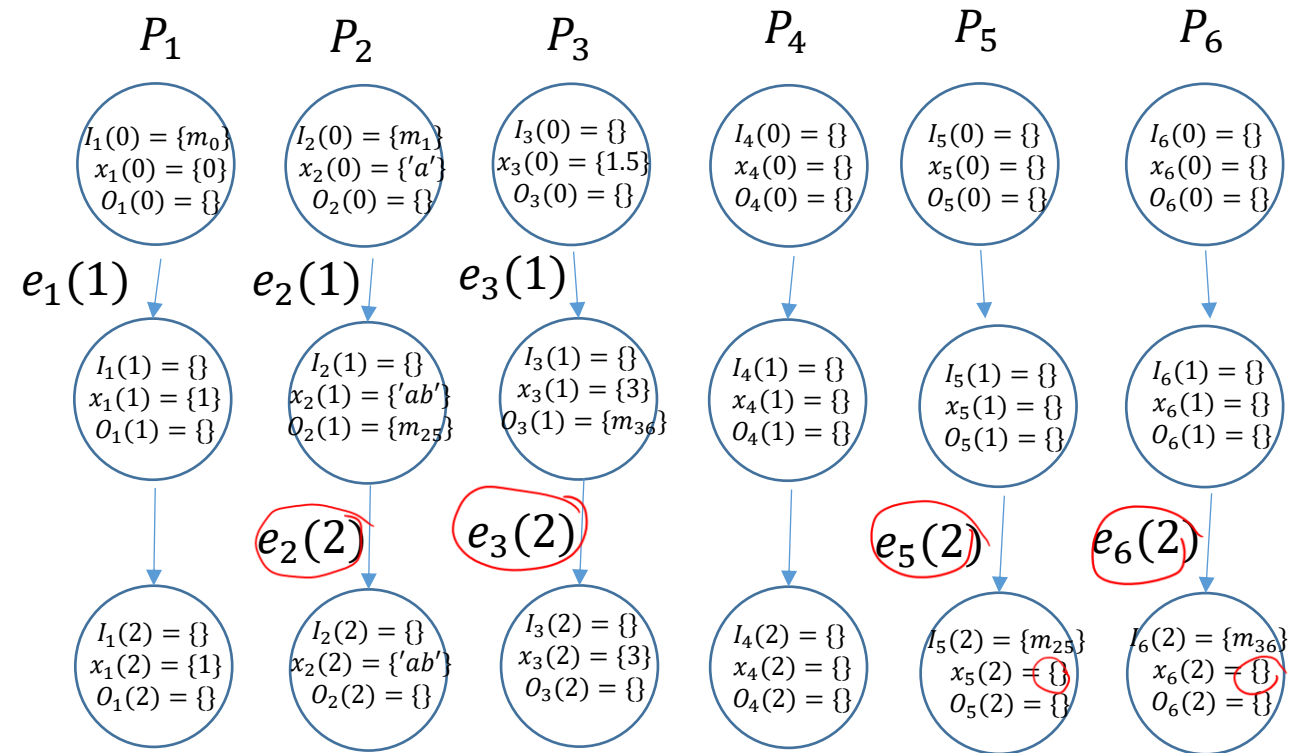
Starea globala a sistemului la momentul t :

$$x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \\ x_3(t) \\ x_4(t) \\ x_5(t) \\ x_6(t) \end{bmatrix}$$

Starea initiala a sistemului:

$$x(0) = \begin{bmatrix} 0 \\ a \\ 1,5 \\ NULL \\ NULL \\ NULL \end{bmatrix} \Rightarrow x(1) = \begin{bmatrix} 1 \\ ab \\ 3 \\ NULL \\ NULL \\ NULL \end{bmatrix}$$

Modelul principal in SD sincron



Modelul principal in SD sincron

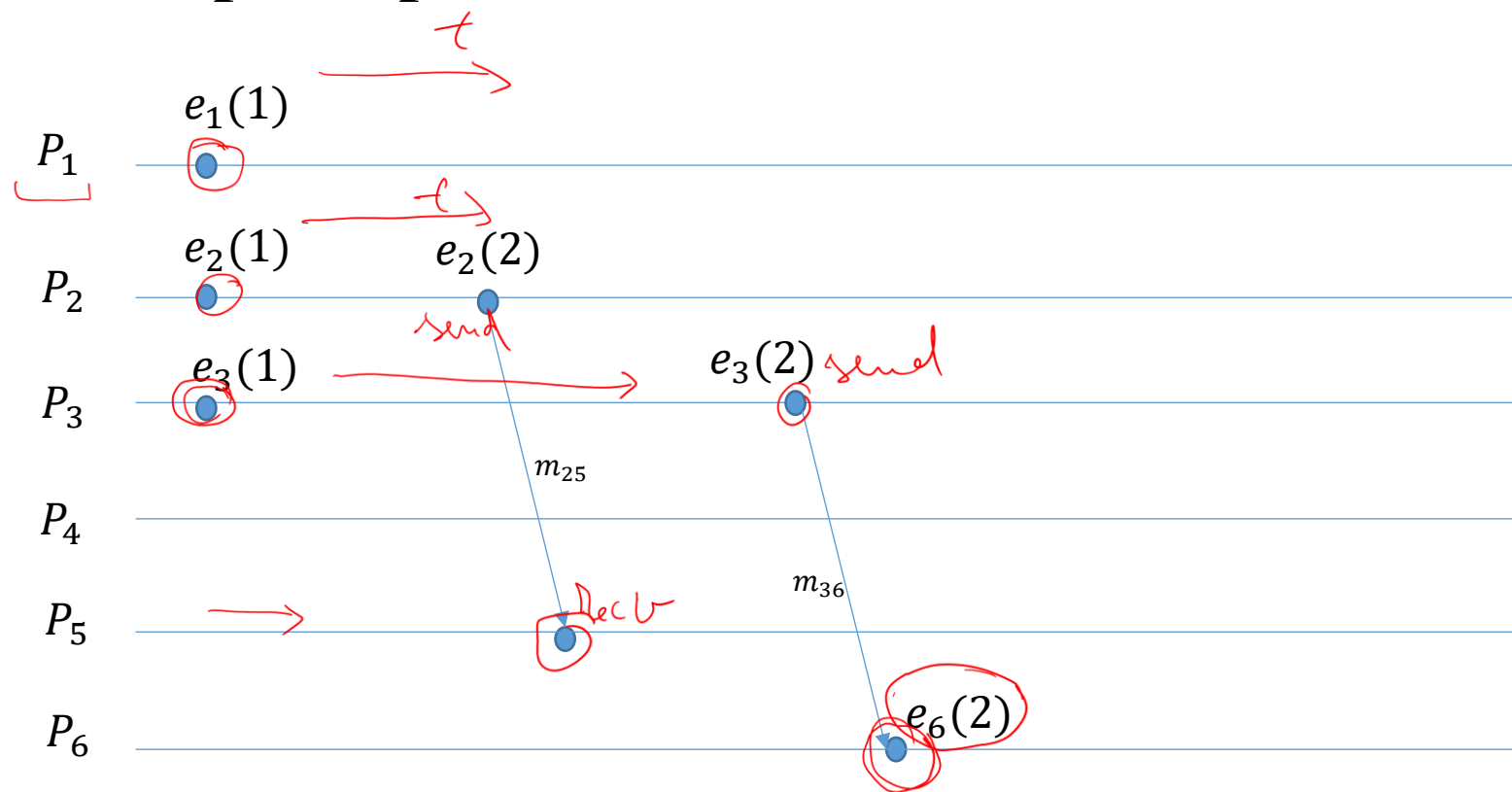


Diagrama spatiu-timp:

- Fiecare proces are propria vedere locala asupra evenimentelor
- Exprima precedenta cauzala intre evenimente

Executie - traectorie

Retele nesigure: posibile pierderi pe comunicatia de mesaje (*packets loss*) sau caderi pe noduri (*crash-faults*). Procesele pornesc din starea initiala $x(0)$.

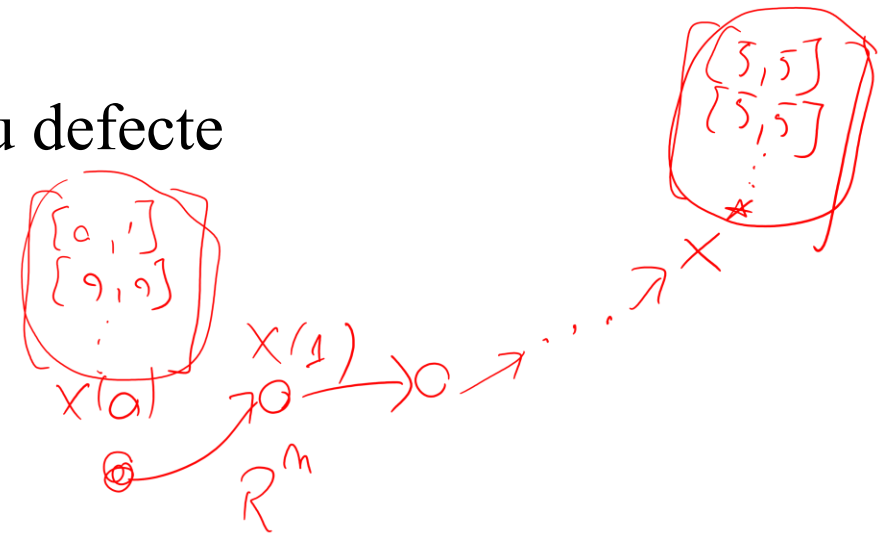
- O traectorie/executie: un sir (in)finit

$$x(0), \underbrace{e(1)}, x(1), \underbrace{e(2)}, x(2), \dots$$

Retele sigure: nu se iau in calcul pierderi de pachete sau defecte

- O traectorie/executie: un sir (in)finit

$$x(0), x(1), x(2), \dots$$



Comunicație

Comunicație

În limbajul natural (uman) exprimarea unui mesaj cuprinde:

- Dialect
- Limbă
- Cuvânt
- Alfabet

Comunicație

Premise necesare pentru comunicație:

- Sursa și destinația sunt conectate fizic
- Sursa alege alfabetul, limba etc., în care codifică mesajul
- Destinația decodifică mesajul (recunoaște alegerile sursei)

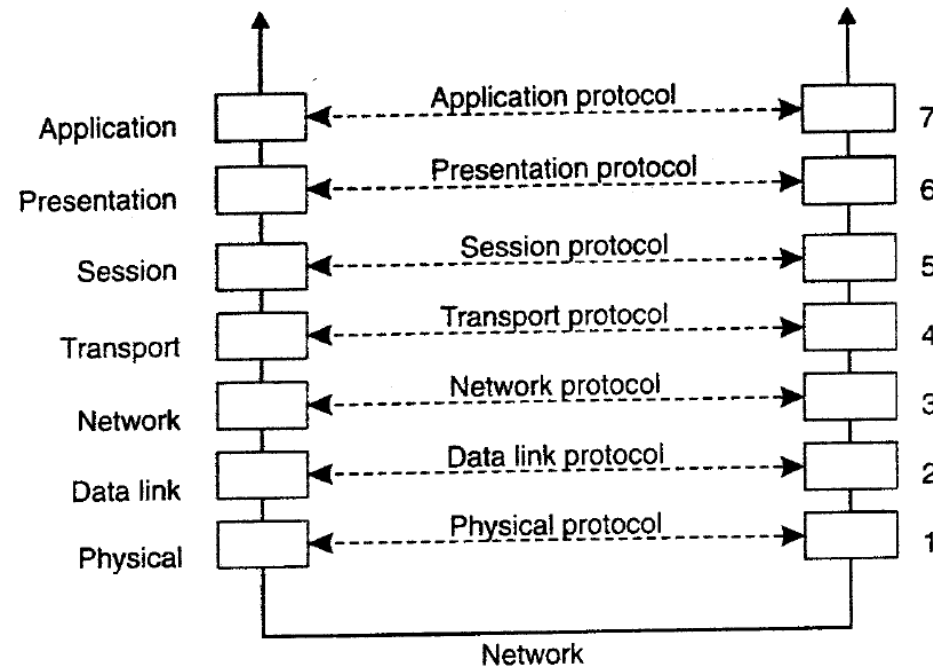
Modele de comunicație

A comunică cu B:

- A produce un mesaj
- Apelează o primitivă pentru a trimite mesajul
- B apelează o primitivă pentru a primi mesajul

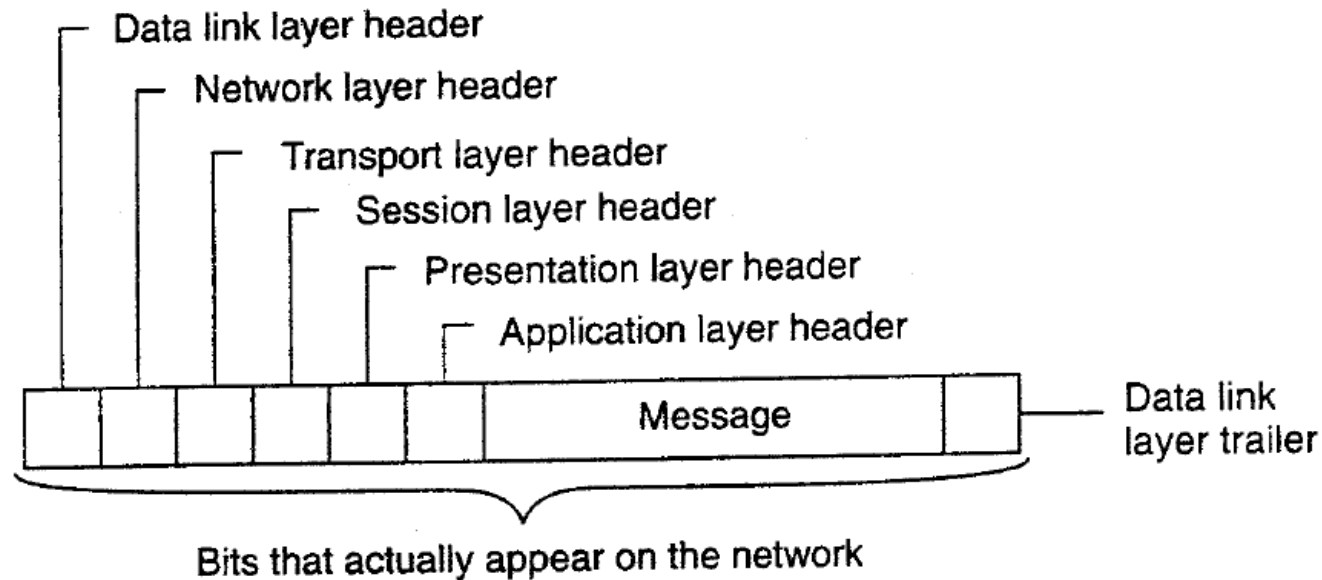
Protocoloale de comunicație

Protocol = algoritm care specifică formatul, conținutul și sensul mesajelor trimise și primite de către un proces.



Protocoloale de comunicație

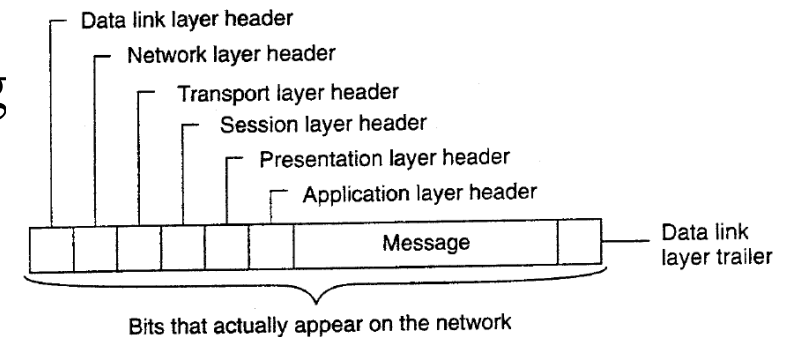
Protocol = algoritm care specifică formatul, conținutul și sensul mesajelor trimise și primite de către un proces.



Ex.: User Datagram Protocol (UDP)

Protocol de comunicație la nivelul (layer) de transport

- Fără conexiune = sursa comunică pachete către destinație fără a negocia o conexiune (implicit fără gestiunea pachetelor)
- Lightweight = nu necesită ordonare sau istoric de pachete; comunicația mesajului începe de la primul pachet
- Singurul mecanism de siguranță este *checksum*, nu există verificarea succesului transmisiei.
- Folosit în aplicațiile de streaming, broadcasting



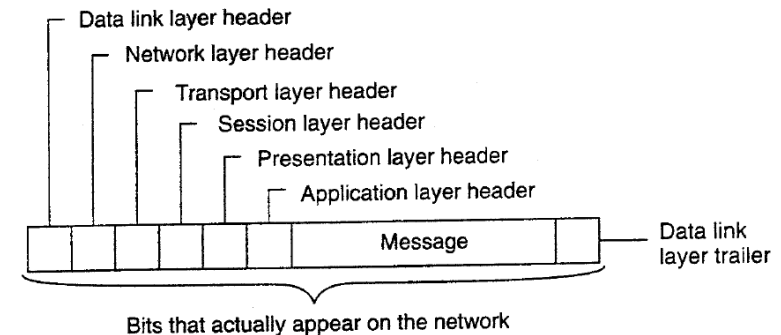
Ex.: User Datagram Protocol (UDP)

Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Source Port															Destination Port																
4	32	Length															Checksum																
8	64	Data																															
12	96																																
⋮	⋮																																

Ex.: Transmission Control Protocol (TCP)

Protocol de comunicație la nivelul (layer) de transport

- Orientat pe conexiune = înainte de interschimbarea de mesaje dintre sursă și destinație, se realizează o conexiune.
- Heavyweight = primele 3 pachete necesare pentru conexiune prin socket.
- Mecanisme de ACK, retransmisie și time-out
- Cuprinde trei etape:
 - Stabilire conexiune
 - Transfer date
 - Opreire conexiune
- Divide mesajul în porțiuni și creează segmente de dimensiuni fixe

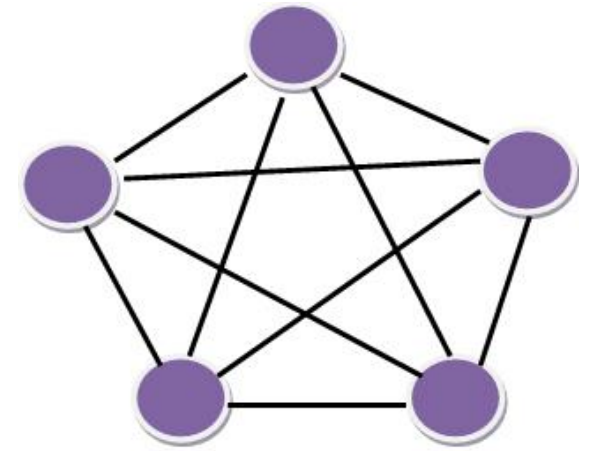
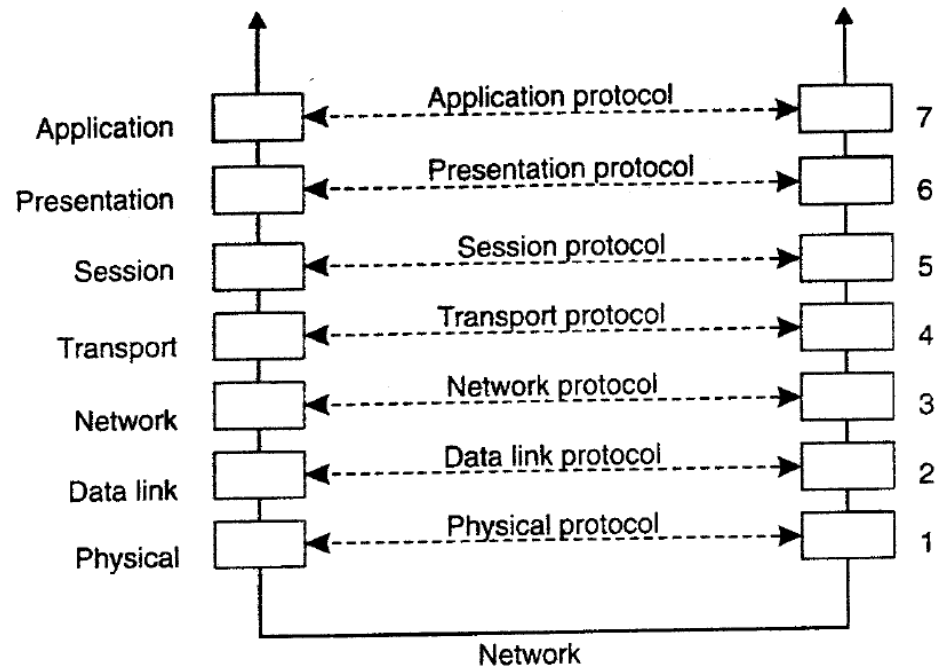


Ex.: Transmission Control Protocol (TCP)

Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Source Port																Destination Port															
4	32	Sequence Number																															
8	64	Acknowledgement Number (meaningful when ACK bit set)																															
12	96	Data Offset				Reserved				C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window															
16	128	Checksum																Urgent Pointer (meaningful when URG bit set) ^[18]															
20	160	[Options] If present, Data Offset will be greater than 5. Padded with zeroes to a multiple of 32 bits, since Data Offset counts words of 4 octets.																															
⋮	⋮																																
56	448																																
60	480	Data																															
64	512																																
⋮	⋮																																

Modele de comunicație

1. *Comunicație prin mesaje*
2. Comunicație prin mesaje persistente
3. Comunicație colectivă



http://4.bp.blogspot.com/_LLvGwISz_4/UMadzzwP7h/AAAAAAAAAHY/2AaHYokPhck/s1600/all+channel.jpg

Referințe

A. D. Kshemkalyani and M. Singhal, *Distributed Computing: Principles, Algorithms, and Systems*, Cambridge University Press, 2008.

M. Raynal, *Distributed Algorithms for Message-Passing Systems*, Springer-Verlag, 2013.

B. Dumitrescu, *Algoritmi de Calcul Paralel*, www.schur.pub.ro, 2001.

V. Eijkhout, E. Chow, R. van de Geijn, *The Science of Computing: The Art of High Performance Computing, volume 1*, 3rd edition 2022. Series reference: <https://theartofhpc.com>.